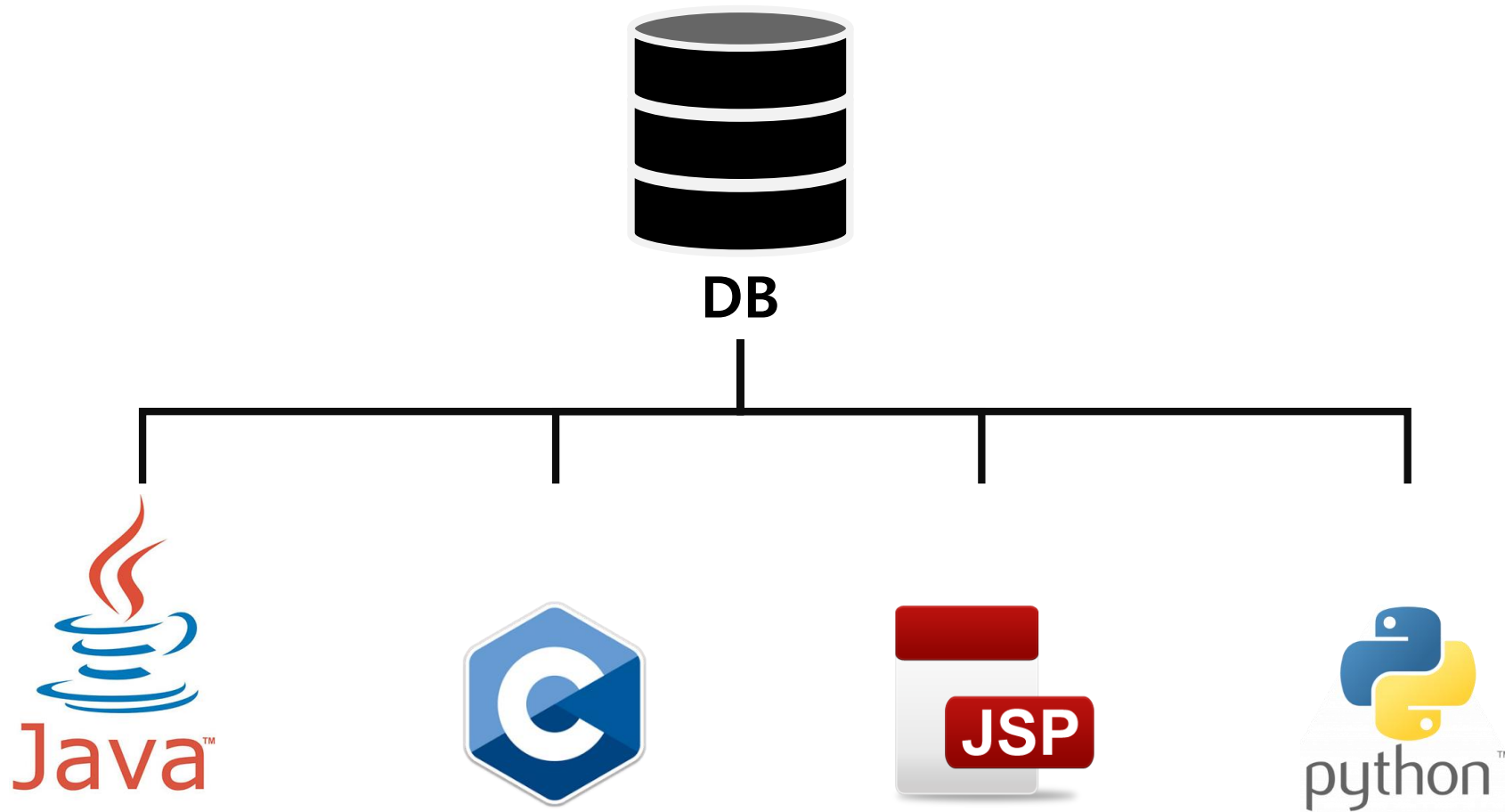


- Data Base 는 데이터를 영구적으로 저장 할 수 있는 저장소 이다.
- Data Base 를 이용하면 서로 다른 시스템 간에 데이터를 공유 할 수 있다.



- Data Base 를 관리하는 시스템을 DBMS(Data Base Management System) 라고 한다.
- 많은 종류의 DBMS 들이 존재한다.



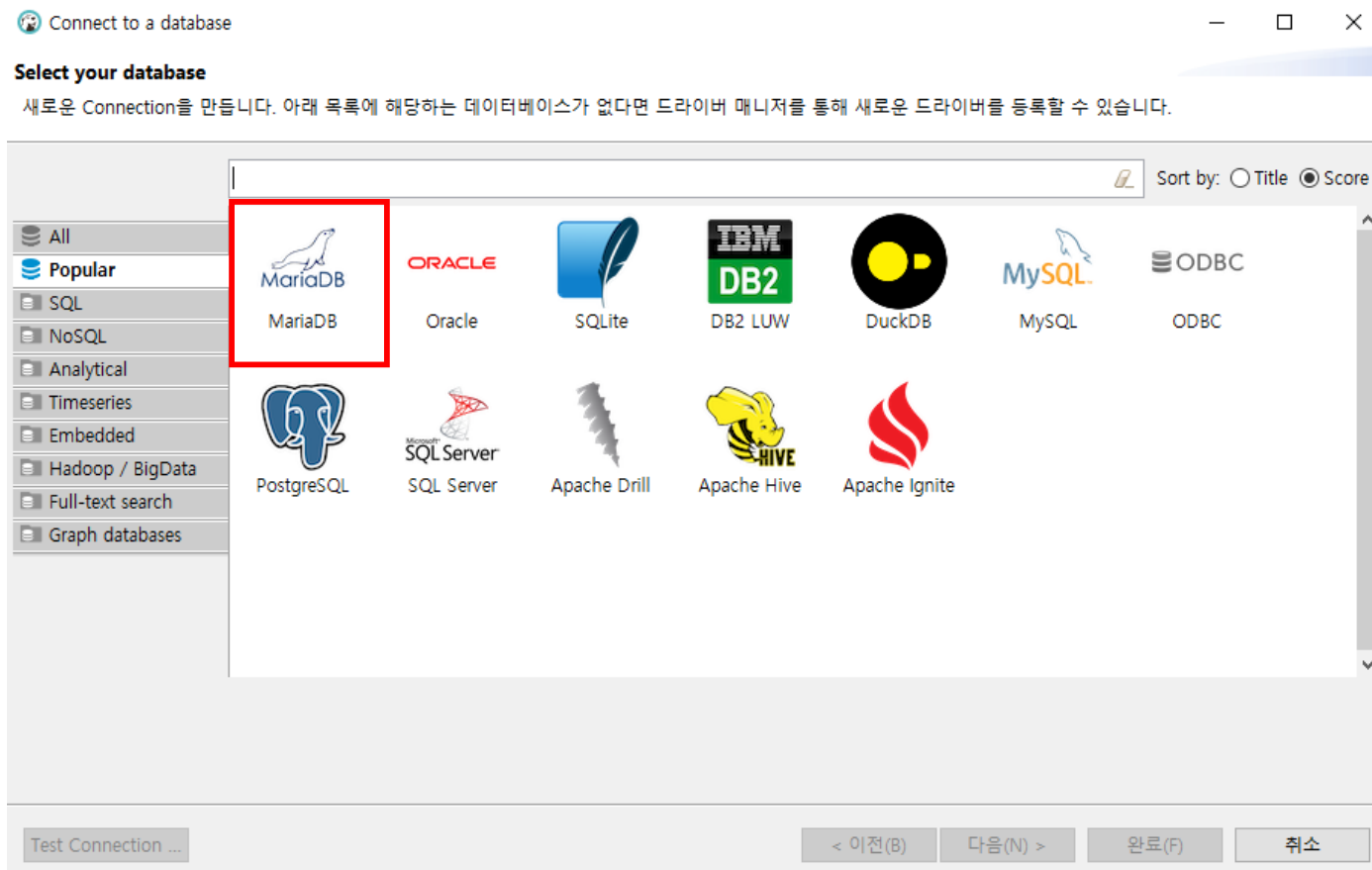
Maria DB

- Data Base Model 은 Relational 과 NO-SQL 로 크게 구분 된다.
- 우리가 배우게 될 Model 은 관계형 데이터 베이스(Relational Data Base) 이다.

Rank			DBMS	Database Model	Score		
Jun 2021	May 2021	Jun 2020			Jun 2021	May 2021	Jun 2020
1.	1.	1.	Oracle +	Relational, Multi-model i	1270.94	+1.00	-72.65
2.	2.	2.	MySQL +	Relational, Multi-model i	1227.86	-8.52	-50.03
3.	3.	3.	Microsoft SQL Server +	Relational, Multi-model i	991.07	-1.59	-76.24
4.	4.	4.	PostgreSQL +	Relational, Multi-model i	568.51	+9.26	+45.53
5.	5.	5.	MongoDB +	Document, Multi-model i	488.22	+7.20	+51.14
6.	6.	6.	IBM Db2	Relational, Multi-model i	167.03	+0.37	+5.23
7.	7.	↑ 8.	Redis +	Key-value, Multi-model i	165.25	+3.08	+19.61
8.	8.	↓ 7.	Elasticsearch +	Search engine, Multi-model i	154.71	-0.65	+5.02
9.	9.	9.	SQLite +	Relational	130.54	+3.84	+5.72
10.	10.	↑ 11.	Microsoft Access	Relational	114.94	-0.46	-2.24
11.	11.	↓ 10.	Cassandra +	Wide column	114.11	+3.18	-4.90
12.	12.	12.	MariaDB +	Relational, Multi-model i	96.79	+0.10	+7.00
13.	13.	13.	Splunk	Search engine	90.27	-1.84	+2.19
14.	14.	14.	Hive	Relational	79.69	+3.51	+1.04

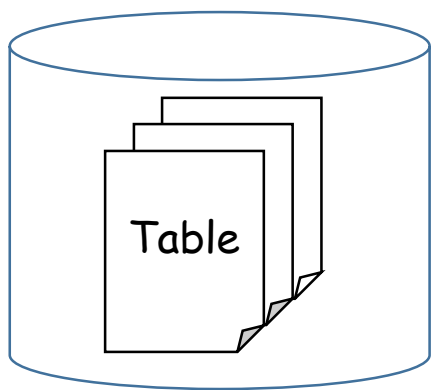
Database 접속

- Dbeaver 라는 SQL 에디터에서 DB 접속을 할 예정 이다.
- 먼저 사용할 Data Base 를 선택 해 준다.

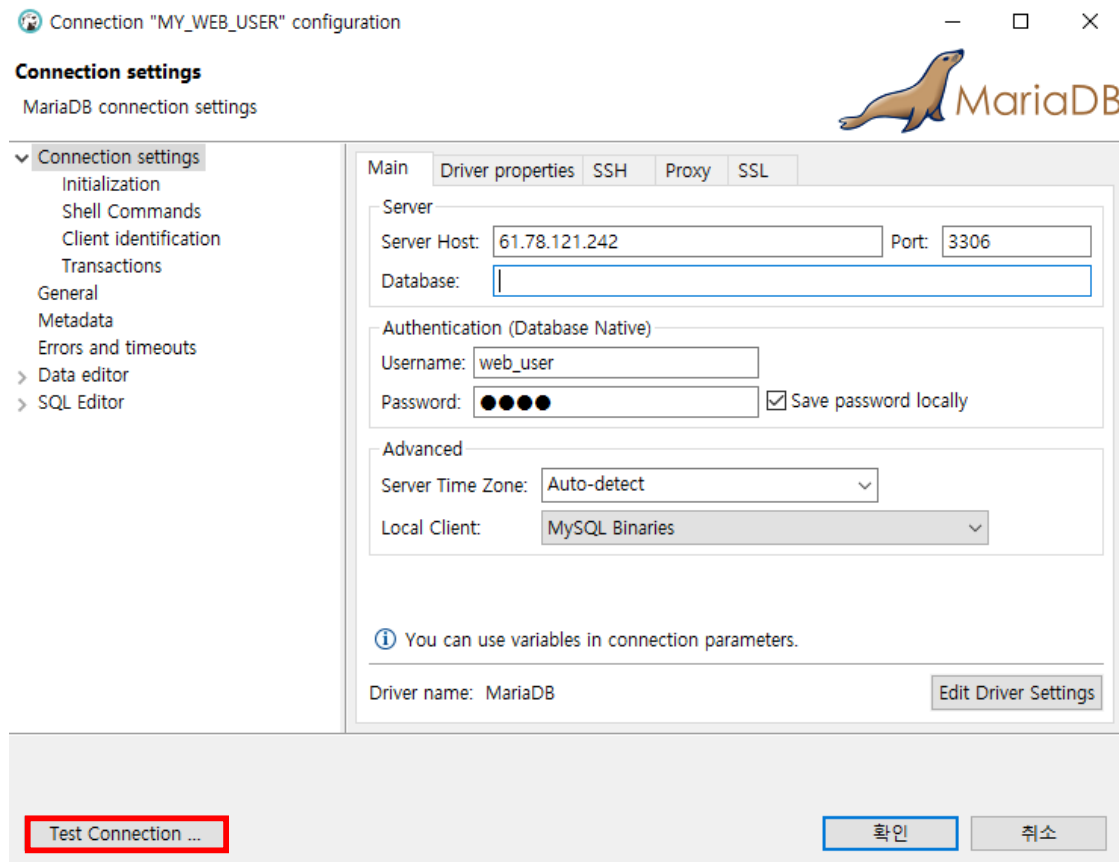


Database 접속

- Mysql 또는 maria db 는 3306 포트를 사용 한다.
- 데이터베이스는 여러 테이블을 저장하는 장소 개념 이다.
- 정보를 넣고 나면 **test connection** 을 해 준다.



database



User 만들기과 Right 생성

- Root 는 모든 권한을 가지고 있다.
- Root 를 통해 사용자 의 명령어 실행 및 접근 권한을 조정 할 수 있다.
- User 생성시 접근 IP 대역을 설정 할 수 있다.

[GRANT | REVOKE] [RIGHT] ON [database.table] FROM [user];

GRANT SELECT, INSERT, UPDATE, DELETE ON *.* TO 'user002'@'192.168.0.111';

부여한다.

읽기,쓰기,수정, 삭제 권한을

모든 데이터베이스 모든 테이블 대상

대상아이디는
user002

01_USER.sql

Table

- 데이터 베이스에서 테이블은 여러 데이터의 집합체 이다.
- JAVA 에서의 *Class* 와 같다고 볼 수 있다

Table

BBS		
키	컬럼명	데이터타입
pk	idx	INT(6)
	user_name	VARCHAR(25)
	subject	VARCHAR(25)
	Content	VARCHAR(500)
	Reg_date	DATE

Class

```
public BBS{  
  
    int idx;  
    String user_name;  
    String subject;  
    String content;  
    Date reg_date;  
  
}
```

Table - 컬럼 및 데이터 다루기

- 현실에서 테이블과 가장 가까운 형태는 Excel 일 것이다.
- 여기서 각 속성은 컬럼 이라고 한다.
- 컬럼은 이름이나 속성을 변경 하거나 추가 삭제도 가능 하다.

idx	user_name	subject	content	reg_date

reg_data

data

```
02_table.sql
```

DML(Data Manipulation Language)

- 데이터를 취급 하는 구문을 우리는 DML 이라고 한다.
- DML 은 SELECT, INSERT, UPDATE, DELETE 문 등이 있다.
- 그리고 key 의 중복 여부에 따라 INSERT 또는 UPDATE 할 수 있는 UPSERT 문이 있다.

Statement	Description
SELECT	특정 조건의 데이터를 조회한다.
INSERT	특정 데이터를 테이블에 삽입 한다.
UPDATE	특정 조건의 데이터를 수정 한다.
DELETE	특정 조건의 데이터를 삭제 한다.
UPSERT	입력되는 key 가 중복일 경우 UPDATE 하고, 아닐 경우 INSERT 한다.

03_DML.sql

SELECT

- 데이터 베이스에서는 특정한 데이터를 **선택(SELECT)** 하여 볼 수 있다.
- 세밀한 선택을 하기 위해서 조건문을 사용 할 수도 있다.

BBS

idx	name	class	level
1	김길동	1반	A
2	나길동	2반	B
3	라길동	3반	C
4	마길동	4반	D
5	송길동	1반	F
6	이길동	2반	A
7	장길동	3반	A
8	차길동	4반	B
9	추길동	2반	B
10	홍길동	3반	A

SELECT name FROM bbs;

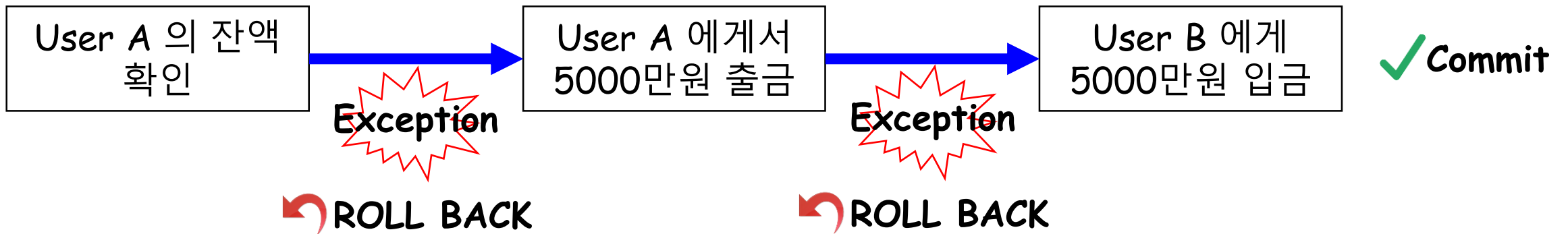
SELECT idx, name, class, level FROM bbs WHERE idx=3;

04_search.sql

TRANSACTION

- Transaction 은 data Base 에서 사용되는 쪼갤 수 없는 업무처리의 단위이다.
- All or Nothing, 하나의 작업 내부의 과정이 잘못되면 전부 취소 되어야 한다.

Internet Banking의 예시(5000 만원 송금)



05_tx.sql

CONSTRAINT

- 제약조건은 컬럼에 데이터를 넣을 경우 특정한 조건을 만족시켜야 할 경우 사용 합니다.

BBS

idx	No	name	class	level
1	171234	김길동	1반	A
2	171235	나길동	2반	B
3	171236	라길동	3반	C
4	171237	마길동	4반	D
5	171238	송길동	1반	F
6	171239	이길동	2반	A
7	171240	장길동	3반	A
8	171241	차길동	4반	B
9	171242	추길동	2반	B
10	171243	홍길동	3반	A

no 는 중복되어서도 값이 없어서도 안 된다.(PRIMARY KEY)

name 은 중복되어도 되지만 값이 꼭 있어야 한다.(NOT NULL)

06_constraint.sql

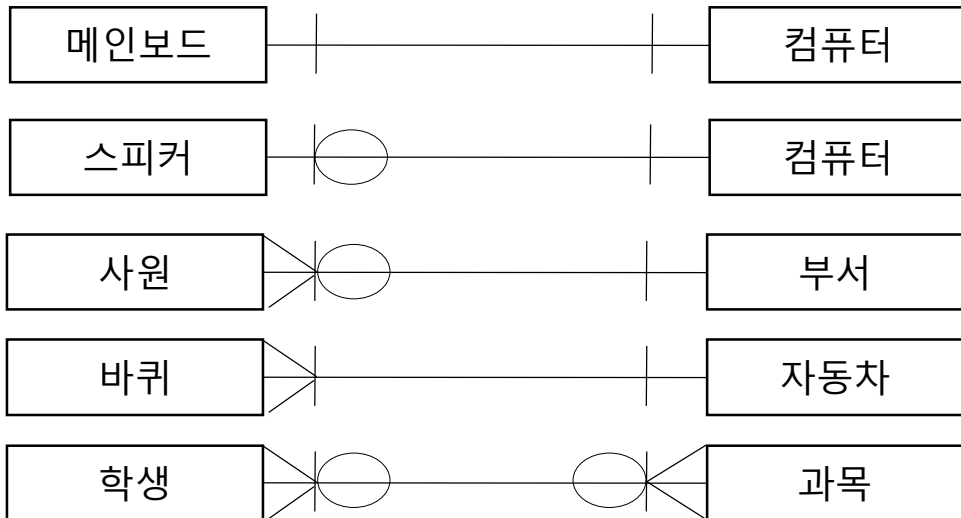
Relation

- RDBMS 의 R 은 Relation 의 약자 이다.
- 그래서 테이블 간에 “부모-자식” 관계 설정을 할 수 있다.

0 : 0개

| : 1개

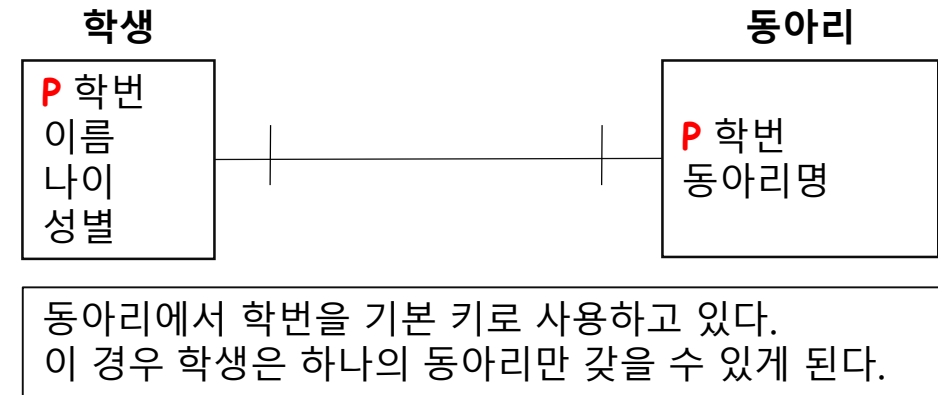
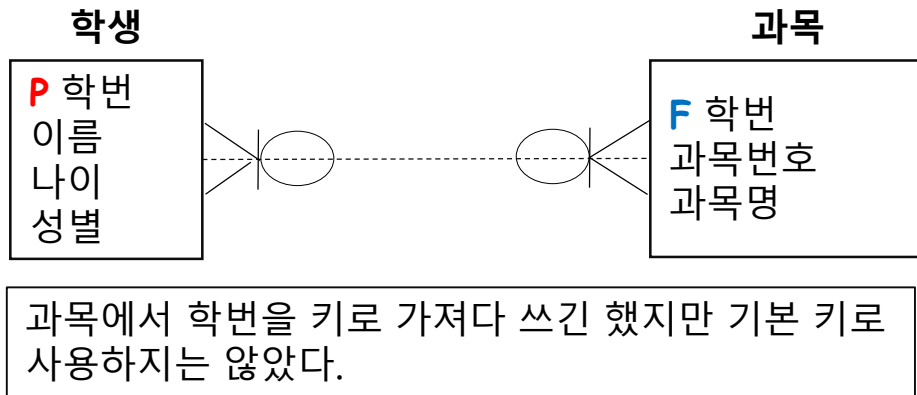
≡ : 1개 이상



- 컴퓨터 한대에 메인보드는 꼭 1개가 있어야 한다.
- 컴퓨터 한대에 스피커 한 개는 있으면 좋은데 없어도 된다.
- 부서에는 하나 이상의 사원이 있을 수도 있지만 없어도 된다.
- 자동차에는 1개 이상의 바퀴가 있어야만 한다.
- 어떻게 설명이 될까요?

Relation

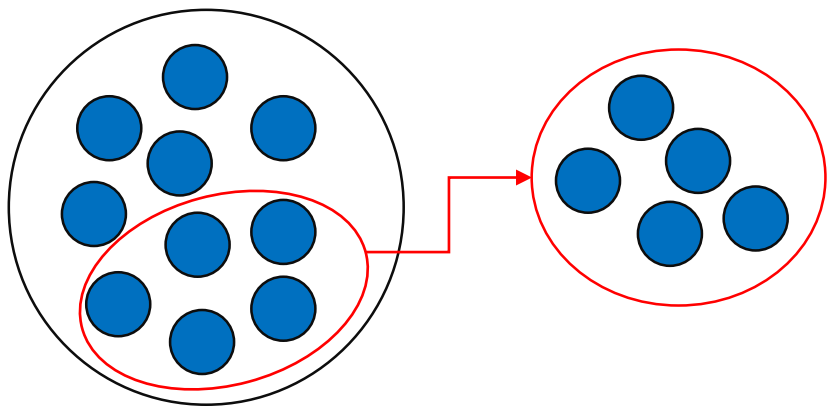
- 이때 연결되는 선은 실선과 점선 에 따라 의미가 달라진다.
- 실선 : **부모의 primary key** 를 **자식**이 primary key 로 사용 또는 포함 (O) <-식별 관계
- 점선 : **부모의 primary key** 를 **자식**이 primary key 로 사용 (X) <-비 식별 관계
- 아래 내용에서 학생 테이블은 과목과 동아리 테이블의 부모 테이블이 된다.



07_reference.sql

Sub Query

- 서브쿼리는 "쿼리 안의 쿼리" 라는 뜻이다.
- 서브쿼리는 사전에 추출된 내용에서 재 검색 하거나, 검색된 내용을 가상 컬럼을 만들어 추가 할 수 있다.
- 즉, 서브쿼리를 사용하는 이유는 가져온 데이터를 재 정제 하기 위함이라 볼 수 있다.



08_subQuery.sql

Join

- 서브쿼리 만을 사용 한다면 쿼리가 굉장히 복잡해 진다.
- 그래서 우리는 join 을 활용 한다.



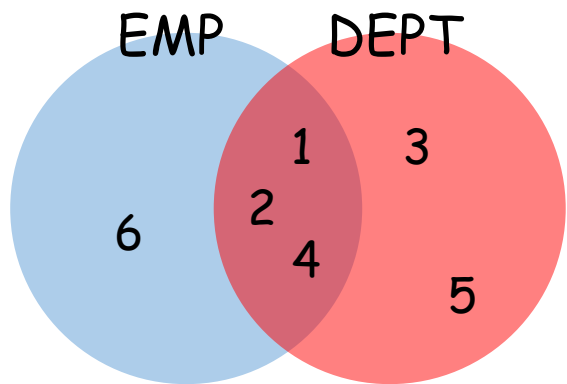
- 특정 사원이 근무하는 팀 이름을 알고 싶다면?
- 일치하는 부서번호의 이름을 가져 오도록 한다.

=> 사원 테이블의 부서번호가 11이면, 팀 테이블의 부서번호가 11인 값을 찾아 그 팀 이름을 찾아 낸다.

09_join.sql

집합(Set)

- 우리는 두 개의 테이블을 집합연산 할 수 있다.
- UNION 은 중복 확인 을 위해 전체 검색 후 정렬하여 검사를 수행 하기 때문에 성능에 좋지 않다.



[1,2,4,6]

[1,2,3,4,5]

UNION(합집합-중복X) : [1,2,3,4,5,6]

UNION ALL(합집합-중복O) : [1,2,4,6,1,2,3,4,5]

INTERSECT(교집합) : [1,2,4]

NOT IN(차집합) : [3,5]

10_Set.sql

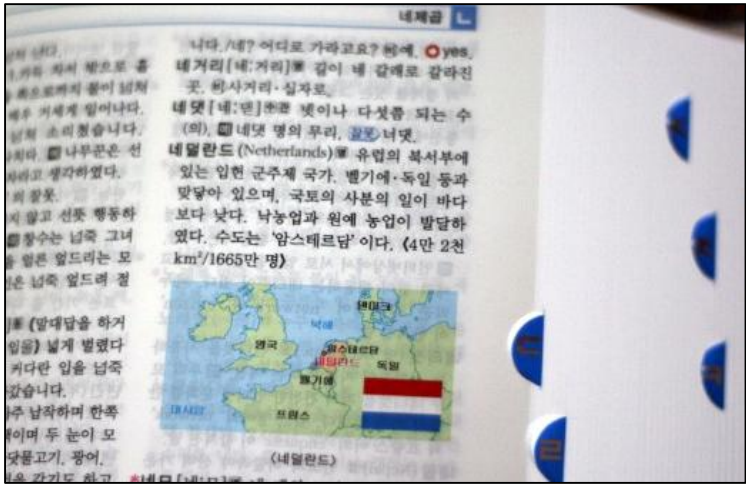
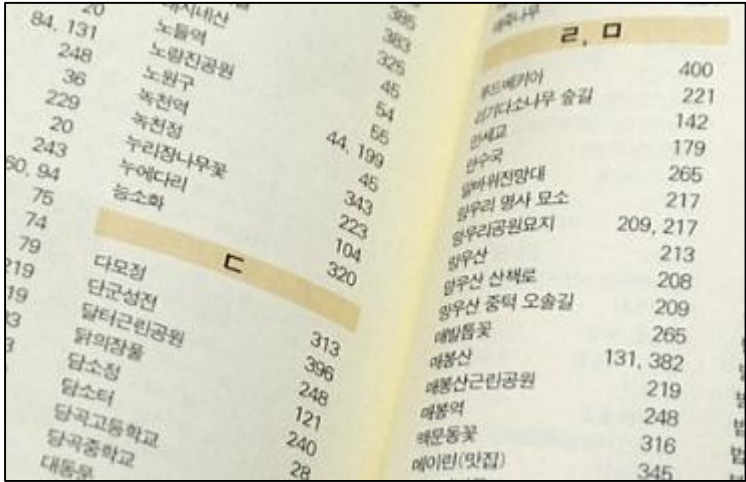
Index

- index(색인) 은 검색을 빠르게 하기 위한 수단이다.
- Primary key 와 Unique key 가 지정되면 따로 index 를 생성된다.

Index 장점	- 검색 속도가 빨라진다. - 시스템의 부하를 줄여 성능 향상.
Index 단점	- 인덱스 생성에 시간과 공간 필요. - 삽입/갱신/삭제 가 빈번히 일어날 경우 성능 저하

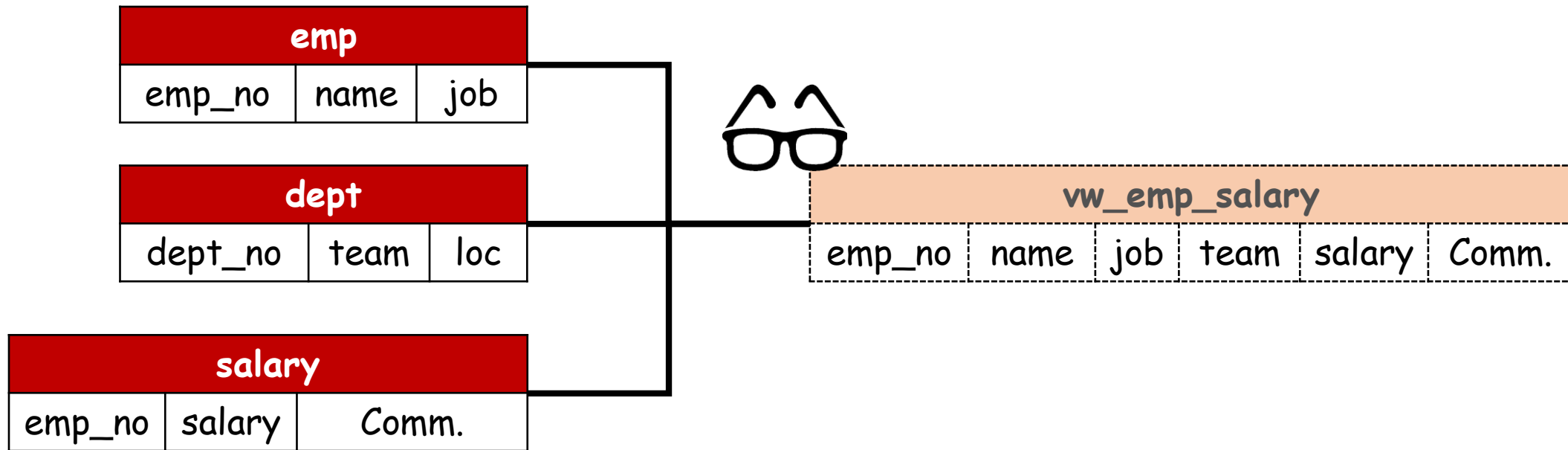
Index 가 필요한 경우	데이터가 많을 경우 JOIN 이 많이 사용된 경우
Index 가 불필요한 경우	데이터가 적을 경우 삽입/갱신/삭제 가 빈번히 일어나는 경우

12_index.sql



View

- 뷰는 하나의 가상 테이블이라 생각 하면 된다.
- 뷰는 복잡한 Query를 통해 얻을 수 있는 결과를 간단한 Query로 얻을 수 있게 한다.
- 한 개의 뷰로 여러 테이블에 대한 데이터를 검색 할 수 있다.



13_view.sql

Auto_increment

- Auto_increment 는 자동으로 증가하는 속성 이다.
- Table 생성시 속성으로 지정해 주거나 이미 생성된 Table 에 추가 해 줄 수 있다.
- 단 auto increment 속성을 사용되는 컬럼은 키 설정이 되어 있어야 한다.

1. 이미 생성된 table 에 설정

```
ALTER TABLE test MODIFY no INT(10) PRIMARY KEY AUTO_INCREMENT;
```

2. Auto increment 초기값 설정

```
ALTER TABLE test AUTO_INCREMENT = 100;
```

14_autoincrement.sql

Limit & offset

- 데이터가 많을 경우 한번에 볼 수 없을 수 있다.
- 그래서 우리는 **limit** 와 **offset** 을 활용 한다.
- 많은 양의 데이터를 **paging** 하여 보여줄 때 유용 하다.

limit n	첫 행부터 n 개만 보여 준다.
limit r, n	r 행 부터 n개만 보여 준다.
limit n offset r	

```
15_limit.sql
```

limit 4, 5
limit 5 **offset** 4

limit 10, 7
limit 7 **offset** 10

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18

Function

- Maria DB(my SQL) 에는 이밖에도 활용 할 수 있는 여러 함수들이 있다.

집계(aggregate) 함수	합계, 최대, 최소, 평균 등을 구하는 함수
문자(character) 함수	문자열을 다루는 함수
날짜(date time) 함수	날짜를 다루는 함수
숫자(numeric) 함수	숫자를 다루는 함수

정규화(normalization)와 역정규화(denormalization)

- RDBMS 설계에서 중복을 최소화하게 데이터를 구조화 하는 프로세스
- 자료 중복으로 인해 이상 현상이 발생 할 수 있어 이를 최소화 하는 것이 정규화의 목적

member	profile	grade
Id	Id	Id
Name	Name	Name
Pw	Hobby	grade

Anomaly	description	Example
삭제 이상	원치 않는 데이터도 삭제	member 의 id 를 삭제하려 했는데 모두 삭제
삽입 이상	원치 않는 데이터가 삽입	profile 의 name 을 넣었는데 grade 에도 삽입
갱신 이상	데이터 일부만 갱신	모든 id 를 변경 하려 했는데 grade 의 id 만 변경

정규화(normalization)와 역정규화(denormalization)

- 이런 정규화(normalization) 과정은 총 여섯 과정이 있다.
- 그 중 실제로는 3NF 까지만 사용 한다.(3NF 가 되면 정규화가 된 것으로 간주 한다.)
- 각 정규형은 이전 단계들을 기본적으로 모두 만족 해야만 한다.

1NF	모든 항목에 값이 있어야 하며(NOT NULL), 중복열이 없어야 한다.
2NF	개체의 속성이 한 식별자에 종속 되어야 한다.(부분 함수적 종속 제거)
3NF	X로 인해 Y가 결정 되고, Y 로 인해 Z 가 결정되는 경우(이행 함수 종속성 제거)

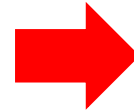
정규화(normalization)와 역정규화(denormalization)

1) 1NF(Normal Form)

- 모든 항목에 값이 있어야 한다.(NOT NULL)
- 중복되는 데이터가 없어야 한다.

<u>id</u>	name	phone1	phone2
123	Kim	01012341234	024568978
456	Hong	01022224545	01022224545
789	Lee	01032324545	✖

- 이 테이블은 null 이 존재한다.
- 그리고 이 경우 전화번호를 2개 이상 넣을 수 없다.
- Hong 의 전화번호는 중복이다.



<u>id</u>	name
123	Kim
456	Hong
789	Lee



<u>id</u>	phone
123	01012341234
456	01022224545
456	01032324545

정규화(normalization)와 역정규화(denormalization)

2) 2NF(Normal Form)

- 부분 함수적 종속성을 제거해야 한다. -> 완전 함수적 종속성
- 복합키의 경우 구성된 일부 컬럼 만으로 식별되면 안된다?

<u>학번</u>	<u>과목번호</u>	학생이름	과목명
1933	1	Kim	Data base
1933	2	Kim	Python
1934	1	Lee	Data base

- 학번+과목번호로 학생이름, 과목명을 알아 낼 수 있다.
- 그런데 학생이름은 학번만으로도 알아낼 수 있다.

정규화(normalization)와 역정규화(denormalization)

2) 2NF(Normal Form)

- 부분 함수적 종속성을 제거해야 한다. -> 완전 함수적 종속성
- 복합키의 경우 구성된 일부 컬럼 만으로 식별되면 안된다?

<u>학번</u>	학생이름
1933	Kim
1934	Lee
1935	park

<u>과목번호</u>	과목명
1	Data base
2	Python
3	Machine learning

학번	과목번호
1933	1
1934	2
1933	1

- 이제 학번을 통해 과목을 구분 할 수 없다.
- 그리고 과목 번호를 통해 학생을 식별 할 수 없다.

정규화(normalization)와 역정규화(denormalization)

3) 3NF(Normal Form)

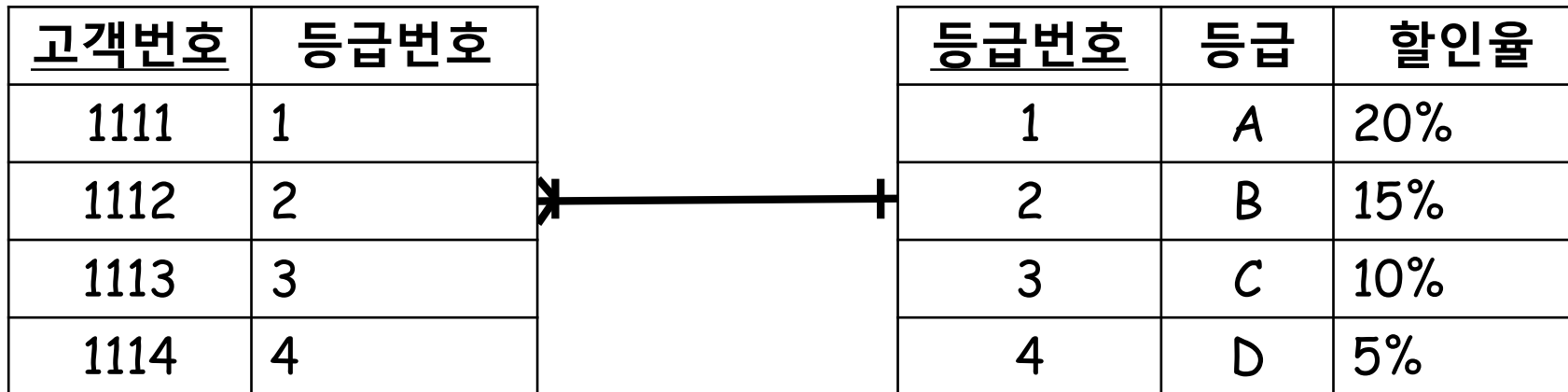
- 고객 번호를 변경하면 그에 따라 등급도 변경해야 하며, 등급에 따라 할인율도 변경 된다.
- 바꿔 말하면 $X(\text{고객번호}) \rightarrow Y(\text{등급})$, $Y(\text{등급}) \rightarrow Z(\text{할인율})$ 일때 $X(\text{고객번호}) \rightarrow Z(\text{할인율})$ 이 되는 것이다.
- 또다른 문제로는 등급에 따른 할인율이 변경 된다면, 얼마나 많은 데이터를 수정해야 하는가?

고객번호	등급	할인율
1111	A	20%
1112	B	15%
1113	C	10%
1114	D	5%

정규화(normalization)와 역정규화(denormalization)

3) 3NF(Normal Form)

- 이렇게 하면 고객번호에 따라 할인율이 결정되는 상황이 사라진다
- 또한 등급에 따른 할인율 변경시 한 곳에서만 데이터를 수정 하면 된다.



정규화(normalization)와 역정규화(denormalization)

- 정규화(normalization) 는 이상 현상을 최소화 하기 위해 테이블을 쪼개는 작업 이다.
- 하지만 지나친 정규화는 쿼리를 복잡하게 만들고 성능을 저하 시킨다.(6개의 테이블을 join 한다면?)
- 그래서 테이블을 합하는 작업을 역 정규화(denormalization)라고 한다.

설계(Design)

- 테이블에 들어갈 값과 제약조건 연관관계를 정의 하는 것을 우리는 모델링 이라고 한다.
- 모델링 한 설계 도면을 우리는 **ERD** 라고 부른다.
- SNS 서비스를 각자 설계 해 보자
- 설계한 내용에 대한 테스트 및 검증이 필요하다.
- 우리가 만든 SNS 서비스의 데이터베이스 에 대해서는 검증 테스트가 필요 할 것이다.

설계(Design)

기 능
회원가입
profile 등록
회원 profile 보기
글 쓰기
reply 쓰기
글 보기(reply 보기)
유저가 작성한 reply 보기
친구목록 보기
좋아요 누른 사람들 보기

새로운 SNS 서비스를 만들려고 한다.

1. 회원가입은 id, 비밀번호, email, 이름을 입력하여 진행 한다.
2. 선택적으로 프로필을 등록할 수 있다.
3. 프로필등록에는 다니는 학교와 회사를 입력해야 한다.
4. 글을 작성 할 수 있다. 간단한 멘트를 남길 수 있다.
5. 이 글에는 등록날짜, 조회 수, 좋아요 숫자를 보여준다.
6. "좋아요" 는 로그인한 회원이 게시글 별로 한번만 가능하다.
7. 해당 글에는 댓글을 달 수 있다.

설계(Design)



설계(Design)

쇼핑몰 서비스를 만들려고 한다.

1. 회원은 일반회원과 판매자 회원이 있다.
2. 일반 회원은 아이디, 비밀번호, 배송지 주소, 연락처 필드가 있다.
3. 판매자 회원은 아이디, 비밀번호, 계좌번호 필드가 있다.
4. 판매 물품에는 품목 아이디, 품목 이름, 가격, 판매자 아이디, 소속 카테고리가 존재 한다.
5. 판매 물품에는 대분류, 중분류, 소분류를 가지고 있다.
6. 카테고리는 대분류, 중분류, 소분류를 고를 때 마다 해당 분류의 물품이 바로 나타나야 한다.
7. 판매 품목은 판매자 회원이 올릴 수 있다.

설계(Design)

8. 구매 테이블에는 사용자아이디, 품목아이디, 품목이름, 가격, 구매 날짜, 수량 등이 저장 된다.
9. 판매자는 본인이 판매한 물품들과 매출을 계산 할 수 있어야 한다.
10. 판매 품목에는 구매후기가 달린다.

설계(Design)

